DocuChat: A Retrieval-Augmented Document Question-Answering Assistant for PDF-Centric Workflows

Doruk Kagan Ergin
Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
78741@office.mans.org.pl

Cuneyt Emre Durak
Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
78745@office.mans.org.pl

Emre Tamer
Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
78713@office.mans.org.pl

Alihan Karaca
Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
78698@office.mans.org.pl

Kumar Nalinaksh
Wydział Zarządzania i Nauk Technicznych
Menedżerska Akademia Nauk Stosowanych w Warszawie
Mazowieckie, Polska
kumar.nalinaksh@office.mans.org.pl

***Abstract*—Long form Pdf document have now become the norm in education research and industry but accessing specific information in them is still time consuming when navigation relies on keyword search and manual reading. Large language model enhance fluency in summarization and question answering whereas standalone generation is limited by the inability to access private documents and by hallucination that are not easily audited.**

This paper proposes DocuChat Pdf based question answering system that uses retrieval augmented generation to ground responses in user supplied documents DocuChat extract and cleans Pdf text divide it into overlapping chunk embeds the chunks into a shared vector space and retrieveal latency trade-offs and evidence-oriented interaction design.

***Index Terms*—Retrieval augmented generation, document question answering, Pdf processing dense retrieval FAISS and evidence grounding.**

I. Introduction

Technical reading required of students and professional has great increased in volume as scientific publications manuals and regulatory text continue to expand in size and revision frequency. Pdf remains a popular delivery format however its structure isnt always convenient for question driven access even when the required information exists in a document the coresponding sentence may be located several pages away from matching keyword.

Keywords search is useful baseline for many tasks but its limited by vocabulary mismatch and a shallow model of context. A query written using synonyms paraphrases or domain specific phrasing may fail to retrieve the relevant paragraph even when the answer is present. Furthermore a number of questions demand synthesis across multiple sections which isnt effectifely supported by single hit keyword matches.

Llms shift document interaction toward natural language question and conversational responses. Two practical problems remain. First a general purpose Llm cant answer questions about a private Pdf unless the document content is provided to the model. Second generation without explicit supporting evidence may produce answers that sound credible but lack documentation which is unacceptable in academic and compliance oriented applications.

In this paper these shortcoming are addressed through the introduction of a RAG pdf question answering assistant called *DocuChat*. The system retrieves relevant passages from the users document and conditions answer generation on that evidence prioritizing traceability over purely fluently output.

The objectives of this work are: to develop an end to end pdfd ingestion and retrieval pipeline suitable for interactive use and to expose evidence in interface to support verification. The main contributions of this paper are a complete RAG workflow for Pdf question answering including ingestion cleaning chunking dense retrieval and evidence conditioned response generation, an interface design that emphasizes auditability through retrieved context display and Pdf excerpt highlighting, and an empirical summary of retrieval accuracy user feedback and latency trends as retrieval depth varies clarifying the performance trade offs of topk retrieval.Paper organization:

Section II reviews related approaches to document retrieval and RAG, Section III summarize practical challenges for Pdf based Qa V describe the proposed solution and implementation Section VI consolidates evaluations outcomes into a unified result section and Sections VII and VIII discus extensions conclude the paper.

II. Literature Review

People often use old method and ask question from Pdf files or to summarize document this kind of task is called long document understanding in many system traditional information retrieval methods are used methods like TF IDFand BM25 are very popular because they are fast and easy to use [1], [2] These method work well when the users question and the document use the same words.But problem start when question use different word.For example document say car but user ask vehicle old system can not understand this.It only match word not meanings becouse of this the system sometimes cannot find the correct answer even if it is present in the document this make system look blind.Another big problem is when answer is not in one place.In many pdf information is spread in many sentence or section.Old retrival method not good in this case.To fix this problem new system use neural retrival method.This method change question and document text into number these number is call vector. [3] Vector try show meaning of text so system not only see word but also idea behind word.Transformer modelhelp a lot for this these model read sentence with context. They look at the words to the left and the right to understanding the meaning [4] [5] dpr use two encoders the one encoder for question one encoder for passages both questions and passage go to same vector space.Because of this system can compare meaning not only word [6] .

in many test, dpr work much better than old sparse method.

But dense retrival also have problem.There can be million of vector. Searching all of them is slow for interactive document assistant and speed is very important user do not want wait long time sosystem use special tool like FAISS help search many vector very fast using Gpu [7] but finding text is still not enough. User want direct answer or short summary. Because of this system also use text generation model.These model can write answer or summary from text [8] [9] . In summarization task ROUGE score is often use ROUGE check how similar generate summary is to correct summary [10] buttext generation model have big problem sometime they write thing that is not true. They can make up information.This is very bad in document system.User want answer that they can check in document.To fix this problem RAG method is use.In RAG system first search document.Then system generate answer only from found text.so system is not free to imagine.It must use evidence [11] .This make answer more correct and less fake.Other system like REALM and RETRO also use retrival inside model.They help the model acquire knowledge from documents or databases [12] , [13] New ideas like Self RAG try to make the system smarter.System learn when need search and when not it also learn how fixed mistake by itself [14].These idea helps make better to Pdf assistant good Pdf assistant not only a give answer it also show where answer come from. For example system show page number or paragraph. So user can open Pdf and check.This make user trust system more. System performance depend on many thing. One important thing is a chunking. Chunking the means cutting document into small part. If chunk is too big system miss detail. If chunk is too small system become slow. Overlap between chunk is also important. Some overlap help keep meaning.

Another important factor retrieval depth which refer how many chunk the system select system usually pick the top K chunks alarger K provide more information but slows down the response, while a smaller K is faster but may miss the answer some systems use crossencoders like BERT to re rank results [14] which improve accuracy but requires more computation ColBERT aims achieve similar quality at a lower cost [15].Long documents are also challenging for standard transformers because they have limited context windows models such as longformer and BigBird address this by using special attention mechanisms to handle longer texts [16], [17] even with longcontext models grounding and source visibility remain important as a user still want to see where answers come from ingeneral effective Pdf question answering and summarization system need strong retrieval find the relevant sections quickly efficient indexing and search and answer based on real evidence with clear source these ideas are exactly what RAG based Pdf assistants follow. a RAG assistant retrieves the top K chunk generates an answer from them and displays the supporting page evidence. this makes the system more transparent and when the system is transparent users tend to trust it more.

III. Challenges

There are a severall to challenges that are especially pertinents pdf based a assistant drivening by prior work and comon use expectation

Semantic mis matching in retrieval lexical technique can fail to locate relevants passage when the a query use a paraphrases or domain specific synonyms [1], [2].

Latency constraints dense a retrieval improves recal but introduce embedding and the vector search overhead which might reduce interactive usability as a retrieval depth increases [5], [6].

Hallucination under the incomplete context without properly retrieved evidence generations models might produce fluenting but unsupprted claim as which motivates evidence grounded generation [7].

Distributed evidence across pages: Many queries require cross section synthesis increases the need for chunking strategies top-k retrieval that preserve context.

User trust and verification: Document assistant are more credible when they provide citations excerpts or highlighting that allow to users to verify the answer directly in the sources.

TABLE I
SUMMARY OF THE KEY METHODS IN DOCUMENTS QUESTION ANSWERING

Studys	Methods	Advantages	Limitations
[1]	Sparse Retrieval TF IDF	Fast and Simple performs well on short queries and small data set	Limited semantics understanding weak for distributed informations
[2]	Sparse Retrieval BM25	Efficient ranking using terms weighting	Lacks contextual semantic understanding
[3]	Dense Retrieval Dpr	Semantic matchings beyond keyword	Higher latency embedding preprocessing required
[4]	Dense Retrieval FAISS	Fast largescale vectors search to with gpu acceleration	Embedding generation is computationally expensive
[5]	LLM Generation GPT	Fluent a natural language generations	Risk of hallucination and unsupported answes
[6]	LLM Generation BERT	Strong contextual embedding and understanding	Limited generative capability
[7]	RAG System	Grounded outputs with retrieval support	Complex systems setup
[8]	RAG System	More accurate and verifiables response	Limited transparency
[9]	Multi Agent Approaches	Specialized agent improves performance	Higher coordination complexity
[10]	Role Specialized Agens	Improved reliabilitys and output qualitys	Requires stricts to workflow design
[11]	Retrievals Augmented Validation	Ensures compliances and grounded result	Depends on external documentation quality
[12]	Iterative Critique and feedback	Reduces errors through iterative refinement	Needs structured monitoring
[13]	Passage Reranking with BERT	Improves top ranked passage accuracy	Additional computational costs
[14]	ColBERT	Good balance between effectiveness and speed	More complex architecture
[15]	BART	Strong summarization and generations performance	Large and resourceintensive
[16]	PEGASUS	Excellent abstractive summarization	Requires large scale pretraining
[17]	ROUGE Evaluation Metric	Standard automatics summarization evaluation	Limited semantic evaluation

IV. PROPOSED SOLUTION

DocuChat is a Pdf based RAG assistant that combine dense retrieval with evidence conditioned response generation in a lightweight web interface.The design objective is not only to provide answer but also to make the basis of each answer visible to the user.

Fig. 1 illustrate the system pipeline a pdf is ingested and a converted into cleaned text. The document is segmented into overlapping chunk to reduce boundary effects when relevant information span adjacent paragraphs. Each chunk is embedded and stored in a FAISS vector index. At query time, the question is embedded into the same space the top$k$ chunks are retrieved and the retrieved text is passed to the LLM with

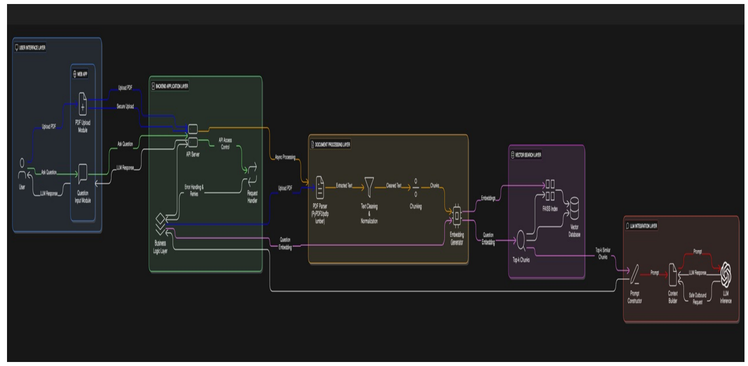

Fig. 1. DocuChat system architecture (logo/watermark to be removed in camera ready diagram export).

a prompt that constrains the response to the provided context metadata is retained to support citation and highlight views.

Fig. 2 summarizes the interaction and inference flow from a user perspective and clarifies how retrieval is integrated into answer generation. the process begins when the user uploads documents and submits a naturallanguage questions through the UI system first processes the questions into an a embedding vector and performs similarity search over the FAISS index to obtain the top k candidate chunks as evidence these retrieved chunks are then a assembled into a bounded context window, ordered and formatted for readability and injected into the Llm prompt so that generation is explicitly conditioned on the retrieved passages rather than in unsupported prior knowledge the Llm produces an answers that is grounded in this evidence context and DocuChat returns both the final response and the supporting chunks to the interface presenting evidence alongside the answer improves transparency and enables rapid verification while also making it easier to diagnose whether an incorrect output was caused by retrieval or generation This workflow therefore operationalizes DocuChat core design principle combining retrieval accuracy with userauditable grounding

The process begins when the user uploads a document and submits a naturallanguage question through the UI the system first processes the question into an embedding vector and performs similarity search over the FAISS index to obtain the top k candidate chunks as evidence. These retrieved chunks are then assembled into a bounded contexts window ordered and formatted for readability and injected into the Llm prompt so that generation is explicitly conditioned on the retrieved passages rather than on unsupported prior knowledge. The LLM produces an answer that is grounded in this evidence context and DocuChat returns both the final response and the supporting chunks to the interface.Presenting evidence alongside the answer improves transparency and enables rapid verification while also making it easier to diagnose whether an incorrect output was caused by retrieval or generation This workflow therefore operationalizes DocuChat core design principle combining retrieval accuracy with userauditable grounding.

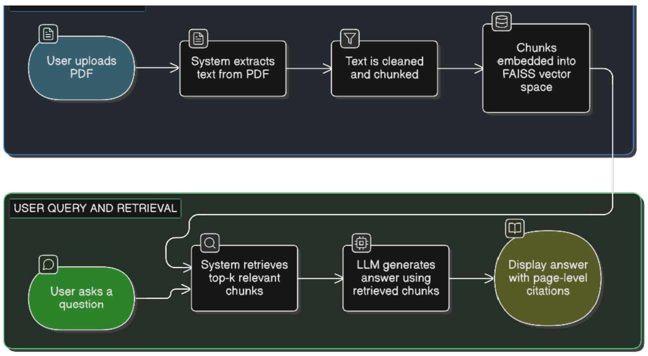

Fig. 2. DocuChat retrieval and generation workflow (logo/watermark to be removed in cameraready diagram export).

Fig. 3 presents the data model used to relate documents chunk embedding and conversational artifacts supporting traceability and potentials persistence. Uploaded Pdfs are represented as document entities that connect to multiple chunk records generated to during preprocessing enabling an onetomany mapping from each source document to its segmented text units each chunk is linked to its corresponding embedding vectors representation which allows the system to retrieve candidates efficiently while still preserving the original chunk text for evidence display on the interaction side user queries and system answers are stored as conversation artifacts enabling sessionlevel history and reproducible evaluation runs crucially the model supports explicit linkage between a specific answers and the set of the chunks retrieved to produce it which enables citationstyle grounding auditing and debugging of system behavior.This relational structure is also suitable for future extensions such as tracking retrieval coverage statistics logging latency per stage storing user feedback on answer quality and maintaining persistent knowledge bases across multiple uploaded documents.

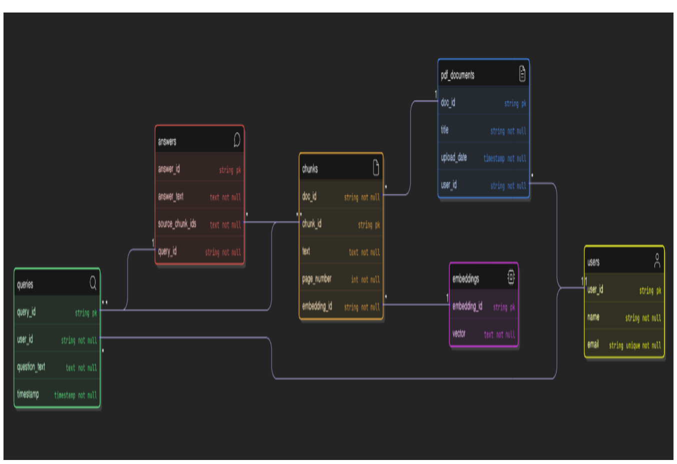

Fig. 3. Entity relationship model for document chunk embeddings querie and answers (logo to be removed in camera ready diagram export).

V. Implementation

The prototype is a deployed as a streamlit applicaition and it has modular pipeline. The pipeline include document ingestion preprocessing, indexing retrieval and response showing the system architecture keep separation between stages this make experimentation and future changes more easy. Each module work independent and communicate with structured data between steps because of this configuration can change without breaking whole system.

Pdf parsing is done with standard python tools. After parsing extracted text is normalize to a remove layout problems like broken lines repeated headers and footers page numbers and bad spacing other preprocessing include fixing whitespace removing non important symbols and trying keep structure when possible so meaning is not losted. These steps help better embedding quality and reduce noise from document format.

Chunking use to overlapping sliding windows with around 500 to 800 tokens per a chunk and 50 to 100 tokens overlap. This help balance between small pieces and keeping enough context. Overlap help reduce information loss and improve retrieval when question is the related to multiple parts of document.

Chunks is converted into to the dense vectors and indexed using a FAISS for the a fast similarity searching question embeddings are calculated in the same embedding space and top k nearest a neighbor retrieval is used evaluation test a different k values to see retrieval sensitivity and answer quality. Retrieved passages are joined with separators and structured prompts before sending to the LLM prompt include clearing instruction to use just only one given context so hallucination and unsupported answers become less.

The evaluation process is done using different test queriesing to the check how the system perform in real scenarios a the questions include both simple keyword search and more descriptive natural languages querie results are observed manually to see if retrieved chunks are relevant and if generated answers stayin inside given context some basics comparison is done between different chunk sizes and overlap settings to understand how configuration affect retrievals quality of overall testing show that modular pipeline make debugging more easy and allow fast iteration when improving system components.

The user interface is designed simple and easy to use so users can be interact withing a system without complex setup the a application streamlit page show different stages of pipeline and allows uploading documents of the running indexing and asking questions in one place retrieved results are displayed together with sources information a so user can be understand where is the answer coming from this also help trust and make validation more easy during testing and development process.

Table II the summarizes a core technologies used to by the prototype in addition to standard queryresponse interaction of the interface supports evidence inspection through contextual display and excerpt highlighting (Fig. 8) this feature allows

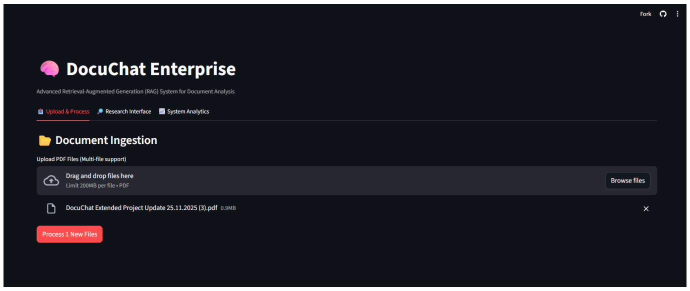

Fig. 4. PDF ingestion view in the Streamlit interfaces.

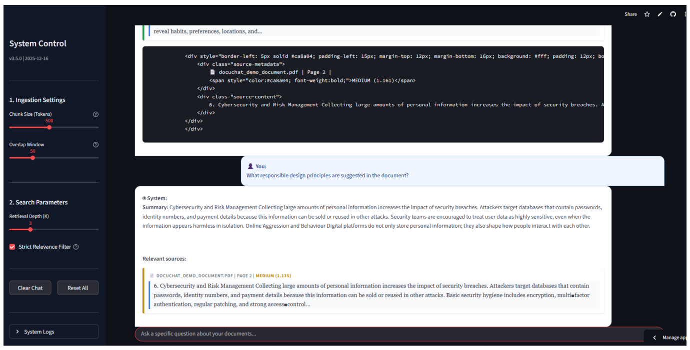

Fig. 5. Embedding and retrieval interface for top k evidence selection.

users to verify the provenance of the generated answers facilitating transparency and strengthening trust in the a system outputs. The designing emphasize explainability to by a exposing retrieved evidence directly within the users interface reinforcing verification a the primary interaction steps.

TABLE II
DOCUCHAT IMPLEMENTATION COMPONENT

Component	Technology	Role in System
Frontend	Streamlit	Provides upload UI chatstyles Q&A, evidence display and export functionality.
Pdf Parsing	`PyPdf2` / `pdfplumber`	Handles text extraction and basics layout normalization.
Chunking	Overlapping token windows	Preserve local coherence and reduce boundary loss.
Embeddings	OpenAI embeddings	maps queries and chunks into a shared vector space
Vector Index	FAISS	Efficient similarity search over chunk embeddings
LLM answering	Openai GPT style model	Generates answers constrained by retrieveed context.
Persistence	SQLite	Stores documents chunks and query metadata for traceability.

VI. RESULTS

The retrieval system demonstrate strong performances for concept search and definition querie achieving

Latency increases approximately linearly with top k. The mean response time rise from 1.2 seconds at k=1k=1k=1 to 5.0 seconds at k=10k=10k=10, reflecting the trade off between evidence coverage and responsivenes. For interactive

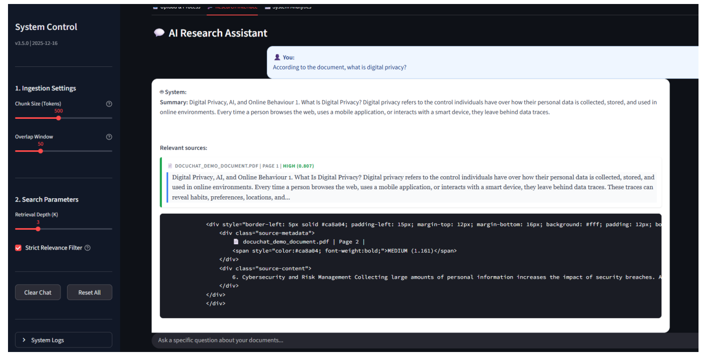

Fig. 6. Answer generation view based on retrieved context.

TABLE III
RETRIEVAL ACCURACY ACROSS QUERY TYPES

Query ID	Query Type	Relevant Chunks Found (%)	Top k
Q01	Concept search	100	3
Q02	Fact look up	90	3
Q03	Multipage reasoning	80	5
Q04	Definition	100	3
Q05	Procedure	85	5

TABLE IV
USER FEEDBACKS (QUALITATIVE COMMENTS)

Participant	Feedback
P1	"Im really trust the result becuse the sources are shown."
P2	"Response are fast and summaries are very clear i recomend!"
P3	"Simple interface chunk adjustment are effective and interesting"
P4	"Transparency is high; citations improve trust."

scenarios moderate value of k=3–5k=3–5k=3–5 provide a practical balance.

User feedback is align with a these observation. Transparency received the highest rating (4.7/5) indicating the importance of evidence visibility. Latency received the lowest rating (4.2/5) consistent with measured response times.

These results suggest that in document centric workflows and auditability is prioritized over minimal latency.

Table III shows the proportion of a case in which relevant chunks were retrieved for five query type. Performance remains high for concept search and definition (

two form of user feedback are reported. Table IV lists qualitative comment from four participant which emphasize evidence visibility and clarity. Table V summarize quantitative score on a 1-5 scale with transparency (4.7) and ease of use (4.6) rated highest notably latency receive the lowest of the four aggregate scores (4.2) aligning with the measured growth in response time a retrieval depth increase. Fig. 7 visualize these rating.

Fig. 8 illustrate the evidence oriented interaction design by linking an answer to highlighted excerpt in the source Pdf.This feature address a central usability concern in LLM based

TABLE V
USER FEEDBACK SUMMARY (1–5 SCALE)

Metric	Score	Comment
Answer accuracy	4.5	Answers perceived as precise and grounded
Transparency	4.7	Page level citation and evidence display improved trust
Ease of use	4.6	Interface perceived as simple and intuitive
Latency	4.2	Response time acceptable for small Pdfs increases with larger contexts

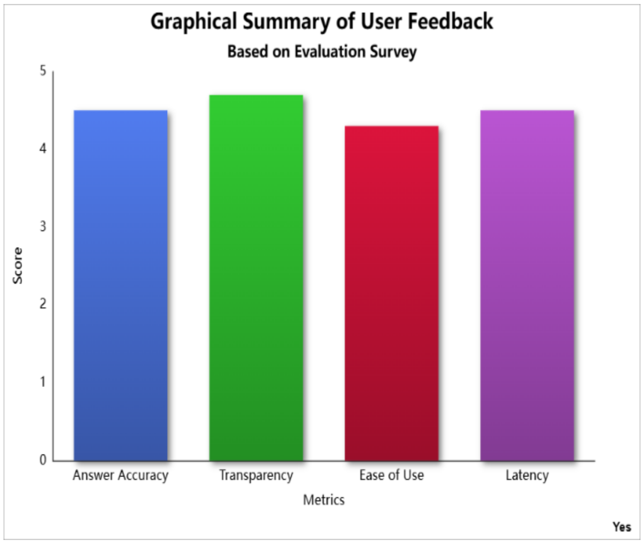

Fig. 7. Graphical summary of user feedback.

TABLE VI
QUERY LATENCY AS A FUNCTION OF RETRIEVAL DEPTH

Top-k	Avg. Latency (s)	Std. Dev.	Comment
1	1.2	0.3	Fast for short question
3	2.4	0.5	Typical interactive setting
5	3.1	0.7	Slower improve coverage on longer Pdf
10	5.0	1.2	Heavy context noticeable delay

assistants: even when an answer is correct user frequently need to confirm it in context, especially for technical definitions and constraints in DocuChat highlighting provides a direct path from generated text to documentary evidence,strengthening the systems auditability.

Table VI and Fig. 9 show that querys latency increases with retrieval depths. The mean latency grows from 1.2 s at k=1 to 2.4 s at k=3, reaching 5.0 seconds at k=10 this behavior reflects an expected trade-off: larger k increases the amount of retrieved evidence and the prompt context passed to the generator, improving recall for distributed evidence at the cost of responsivenes for interactive use, the results suggest that moderate k values offer a reasonable compromise between evidence coverage and perceived speed.

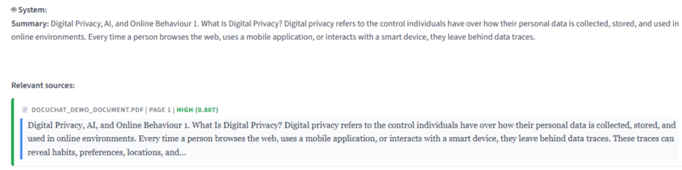

Fig. 8. Example of excerpt display and highlighting within the Pdf view.

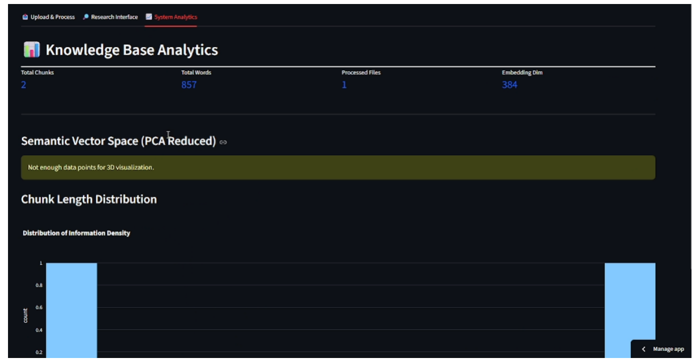

Fig. 9. Latency trend as retrieval depth (top$k$) increases.

VII. FUTURE WORK

Local improvements is could be increase robustness and privacy. Multidocument retrieval would enable cross document question answering a hybrid sparse dense pipeline can improve recall under vocabulary mismatch while lightweight crossencoder re ranking can be raise precision when top-k contains semantically close but irrelevant chunks replacing cloud embeddings and generation with local models would reduce API costs and the strengthen privacy guarantees evidence features could add sentencelevel citations clickable page links and an a evidence panels listing the exact chunks used larger stratified evaluations with relevance labels would better test generalization.

VIII. CONCLUSION

DocuChat is a RAG assistant that extracts text chunks it embeds content and retrieves passages via FAISS to generate evidencegrounded answers a Streamlit UI shows context and highlights source excerpt experiments report 80-100 % relevant chunk coverage across five query types (mean 91%). users praised transparency and usability but higher top k increased latency. Limitations is include a small scope unclear relevance labels and reliance on the external API motivating hybrid retrieval reranking and local models.

Acknowledgment

All author helped write the paper and approve the final version there was no funding the author have no conflicts of interest the code and implementation resources are available on GitHub repositories listed inproject material.

1) **Funding:** Not applicable.
2) **Conflict of Interest:** None.
3) **Ethics Approval and Consent to Participate:** Not applicable.
4) **Consent for Publication:** All author have provided consent.
5) **Data Availability:** Prototype evaluation artifact include a demonstration repository and an informals feedback summary; rawsurvey responses are not publiclys released.
6) **Code Availability:** The sourcecode is available at https://github.com/cuneyted and https://github.com/doruk3535.

References

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *arXiv preprint* arXiv:1702.08734, 2017.

[6] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-augmented language model pre-training," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020.

[9] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, *et al.*, "Improving language models by retrieving from trillions of tokens," in *Proc. ICML*, 2022, pp. 2206–2240.

[10] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint* arXiv:2310.11511, 2023.

[11] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint* arXiv:2004.05150, 2020.

[12] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big Bird: Transformers for longer sequences," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[13] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint* arXiv:1901.04085, 2019.

[14] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR)*, 2020.

[15] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. ACL*, 2020, pp. 7871–7880.

[16] J. Zhang, Y. Zhao, M. Saleh, and P. Liu, "PEGASUS: Pre-training with extracted gap-sentences for abstractive summarization," in *Proc. ICML*, 2020, pp. 11328–11339.

[17] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, 2004, pp. 74–81.